



Cours 02 – Notion 6 : Classes Immuables

2.5.1 Définitions

Des classes immuables permettent de créer des objets qui, une après l'assignation d'une première valeur à leurs attributs, ne peuvent pas changés.

L'utilité de ces classes est de représenter des objets qui une fois qu'ils sont devenus, ne devrait pas pouvoir être modifiés. Parce qu'exemple, si on prend votre certificat de naissance, les données qui y sont conservées ne peuvent être modifiées (généralement parlant, à moins de procéder à un changement de nom légal).

L'emploi du modificateur *final* en conjonction avec des attributs permet généralement de créer ce genre de classe.

2.5.2 Déclaration

Pour déclarer une classe immuable, certaines règles doivent être respectées.

- La déclaration de la classe doit être *final*
- Les attributs de la classe sont *privés* et *finals*.
- Tous les attributs d'une classe sont initialisés via le constructeur par une copie de leur contenu (surtout en ce qui a trait aux objets composés).
- Les accesseurs retournent des copies des objets pour s'assurer que le contenu de la classe immuable ne puisse être modifié par référence.

Avant de débiter la déclaration d'une classe immuable (CertificatNaissance.java) considérons, l'existence d'une classe ordinaire (ou d'une classe mutable) qui représente une date de calendrier :

```
public class DateNaissance
{
    private int jours;
    private int mois;
    private int annee;

    public DateNaissance(int jours, int mois, int annee)
    {
        this.jours = jours;
        this.mois = mois;
        this.annee = annee;
    }

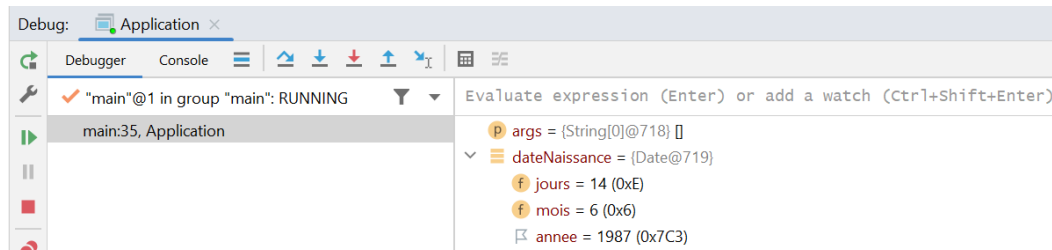
    void setJours(int jours)
    {
        this.jours = jours;
    }

    void setMois(int mois)
    {
        this.mois = mois;
    }

    void setAnnee(int annee)
    {
        this.annee = annee;
    }
}
```

Une fois qu'une date a été créé, il est facile de modifier l'un de ces attributs par l'emploi des mutateurs.

```
public class Application
{
    public static void main(String[] args)
    {
        Date dateNaissance = new Date(14, 06, 2022); //Suis-je né en 2022?
        dateNaissance.setAnnee(1987); //Correction de l'année de naissance
    }
}
```



L'exécution de cette application démontre la mutabilité de la classe Date.

Démontrons la déclaration d'une classe immuable en gérant un certificat de naissance. La démonstration des points a) et b) se fait de la manière suivante :

```
import java.util.Objects;

final public class CertificatNaissance
{
    final private String nom;
    final private String lieuNaissance;
    final private DateNaissance dateNaissance;
    final private long noIdentification;

    public CertificatNaissance(String nom, String lieu, DateNaissance date)
    {
        this.nom = nom;
        this.lieuNaissance = lieu;
        this.dateNaissance = date;

        //Les certificats de naissances possède un no unique.
        //La fonction de hachage permet de créer se numéros.
        this.noIdentification = Objects.hash(nom, lieu, date, java.time.LocalDate.now());
    }
}
```

b) La classe est *final*.

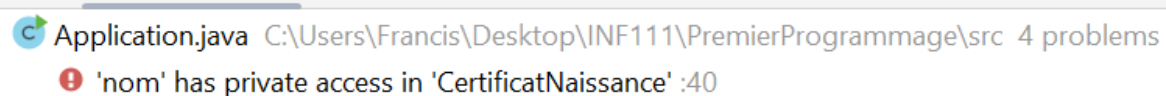
a) Les attributs sont finaux et privés.

On remarque dans le "main", que les attributs ne sont pas accessibles et comme il n'y a pas de méthode mutatrice, il n'est pas possible de modifier leur valeur.

```
import java.util.Date;
public class Application
{
    public static void main(String[] args)
    {
        DateNaissance maNaissance = new DateNaissance(14, 06, 1987);

        CertificatNaissance moi = new CertificatNaissance("Francis Bourdeau",
                                                            "Repentigny",
                                                            maNaissance);

        moi.nom = "Big Franky";
    }
}
```



Comme il n'y a pas de méthode mutatrice d'offerte, il est impossible de modifier les attributs que sont de types primitifs. On conclut que les critères a) et b) sont respectés.

Toutefois, le constructeur a conservé une référence de l'objet `DateNaissance` et cet objet existe encore dans l'application. Puisque cette classe n'est pas immuable, une modification à la date de naissance dans le "main" entre une modification à l'objet immuable.

```

28 import java.util.Date;
29 public class Application
30 {
31     public static void main(String[] args)
32     {
33         DateNaissance maNaissance = new DateNaissance(2022, 06, 1987);
34         CertificatNaissance moi = new CertificatNaissance("Francis Bourdeau",
35                                                         "Repentigny",
36                                                         maNaissance);
37
38
39         //Rajeunisson de 10 ans.
40         maNaissance.setAnnee(1997);
41     }
42 }

```

À la suite de l'exécution de la ligne 34, on voit que le certificat de naissance est émis pour une personne née en 1987. Toutefois après l'exécution de la ligne 40, l'objet a changé même si nous ne l'avons pas affecté directement.

Après l'exécution de la ligne 34 :	Après l'exécution de la ligne 40 :
<pre> > moi.dateNaissance = (DateNaissance@720) > args = (String[0]@719) [] > maNaissance = (DateNaissance@720) > moi = (CertificatNaissance@922) > f nom = "Francis Bourdeau" > f lieuNaissance = "Repentigny" > dateNaissance = (DateNaissance@720) f jours = 2022 (0x7E6) f mois = 6 (0x6) f annee = 1987 (0x7C3) f noIdentification = 1318235449 (0x4E92AD39) </pre>	<pre> > moi.dateNaissance = (DateNaissance@720) > args = (String[0]@719) [] > maNaissance = (DateNaissance@720) > moi = (CertificatNaissance@922) > f nom = "Francis Bourdeau" > f lieuNaissance = "Repentigny" > dateNaissance = (DateNaissance@720) f jours = 2022 (0x7E6) f mois = 6 (0x6) f annee = 1997 (0x7CD) f noIdentification = 1318235449 (0x4E92AD39) </pre>

Ce principe se nomme le "shallow copie". Simplement conserver une référence sur un objet est un risque important de bogue logique, très difficile à détecter. Pour régler la situation, le constructeur doit créer un nouvel objet avec les données de l'objet reçu.

<pre> public class DateNaissance { private int jours; private int mois; private int annee; public DateNaissance(int jours, int mois, int annee) { this.jours = jours; this.mois = mois; this.annee = annee; } public DateNaissance(DateNaissance obj) { this.jours = obj.jours; this.mois = obj.mois; this.annee = obj.annee; } void setJours(int jours) { this.jours = jours; } void setMois(int mois) { this.mois = mois; } void setAnnee(int annee) { this.annee = annee; } } </pre>	<pre> import java.util.Objects; final public class CertificatNaissance { final private String nom; final private String lieuNaissance; final private DateNaissance dateNaissance; final private long noIdentification; public CertificatNaissance(String nom, String lieu, DateNaissance date) { this.nom = nom; this.lieuNaissance = lieu; this.dateNaissance = new DateNaissance(date); long no = Objects.hash(nom, lieu, date, java.time.LocalDate.now()); this.noIdentification = no; } } </pre>
--	---

La construction d'une nouvelle date à partir de celle reçue en paramètre s'appelle un "deep copy" et comme l'objet immuable contient maintenant une référence différente de l'objet mutable du "main", une modification à celui-ci n'entraîne plus la modification de la classe immuable.

```

28 import java.util.Date;
29 public class Application
30 {
31     public static void main(String[] args)
32     {
33         DateNaissance maNaissance = new DateNaissance(2022, 06, 1987);
34         CertificatNaissance moi = new CertificatNaissance("Francis Bourdeau",
35                                                         "Repentigny",
36                                                         maNaissance);
37
38
39         //Rajeunisson de 10 ans.
40         maNaissance.setAnnee(1997);
41     }
42 }

```

À la suite de l'exécution de la ligne 34, on voit que le certificat de naissance est émis pour une personne née en 1987. Après l'exécution de la ligne 40, l'objet n'a pas changé.

Après l'exécution de la ligne 34 :	Après l'exécution de la ligne 40 :
<pre> > moi.dateNaissance = {DateNaissance@720} > args = {String[0]@719} [] > maNaissance = {DateNaissance@720} > moi = {CertificatNaissance@922} > f nom = "Francis Bourdeau" > f lieuNaissance = "Repentigny" > f dateNaissance = {DateNaissance@720} f jours = 2022 (0x7E6) f mois = 6 (0x6) f annee = 1987 (0x7C3) f noldentification = 1318235449 (0x4E92AD39) </pre>	<pre> > args = {String[0]@919} [] > maNaissance = {DateNaissance@920} > moi = {CertificatNaissance@921} > f nom = "Francis Bourdeau" > f lieuNaissance = "Repentigny" > f dateNaissance = {DateNaissance@922} f jours = 2022 (0x7E6) f mois = 6 (0x6) f annee = 1987 (0x7C3) f noldentification = 1318235450 (0x4E92AD3A) </pre>

Nous venons donc de démontrer l'importance de créer de vraies copies lors de la création d'un objet tel que requis par la spécification c).

Finalement, la même problématique de "shallow copy" apparaît lorsque l'on consulte un sous-objet d'une classe immuable par l'entremise d'un accesseur.

```

29 public class Application
30 {
31     public static void main(String[] args)
32     {
33         DateNaissance maNaissance = new DateNaissance(2022, 06, 1987);
34         CertificatNaissance moi = new CertificatNaissance("Francis Bourdeau",
35                                                         "Repentigny",
36                                                         maNaissance);
37
38
39         //Rajeunisson de 10 ans.
40         DateNaissance d = moi.getDate();
41         d.setAnnee(1997);
42     }
43 }

```

À la suite de l'exécution de la ligne 34, on voit que le certificat de naissance est émis pour une personne née en 1987. Une modification à l'objet retournée entraîne une modification non-désiré à l'objet immuable.

Après l'exécution de la ligne 34 :	Après l'exécution de la ligne 41 :
<pre> > moi.dateNaissance = {DateNaissance@720} > args = {String[0]@719} [] > maNaissance = {DateNaissance@720} > moi = {CertificatNaissance@922} > f nom = "Francis Bourdeau" > f lieuNaissance = "Repentigny" > f dateNaissance = {DateNaissance@720} f jours = 2022 (0x7E6) f mois = 6 (0x6) f annee = 1987 (0x7C3) f noldentification = 1318235449 (0x4E92AD39) </pre>	<pre> > args = {String[0]@920} [] > maNaissance = {DateNaissance@921} > moi = {CertificatNaissance@922} > f nom = "Francis Bourdeau" > f lieuNaissance = "Repentigny" > f dateNaissance = {DateNaissance@925} f jours = 2022 (0x7E6) f mois = 6 (0x6) f annee = 1997 (0x7CD) f noldentification = 1318235450 (0x4E92AD3A) > d = {DateNaissance@925} </pre>

La même stratégie doit être employée et l'accesseur aux objets doivent retourner une copie d'un nouvel objet, avec le même contenu. Donc, voici le code de la classe `CertificatNaissance.java` qui respecte les critères d'une classe immuable.

```
import java.util.Objects;

final public class CertificatNaissance
{
    final private String      nom;
    final private String      lieuNaissance;
    final private DateNaissance dateNaissance;
    final private long        noIdentification;

    public CertificatNaissance(String nom, String lieu, DateNaissance date)
    {
        this.nom = nom;
        this.lieuNaissance = lieu;
        this.dateNaissance = new DateNaissance(date);
        this.noIdentification = Objects.hash(nom, lieu, date, java.time.LocalDate.now());
    }

    DateNaissance getDate()
    {
        return new DateNaissance(this.dateNaissance);
    }

    String getNom()
    {
        return this.nom;
    }

    String getLieuNaissance()
    {
        return this.lieuNaissance;
    }

    long getNoIdentification()
    {
        return this.noIdentification;
    }
}
```

a) La classe est *final*.

b) Les attributs sont finaux et privés.

c) "Deep copy" des objets dans le constructeur

d) "Deep copy" des objets dans les accesseurs.